

VarBlockSpMM: A Shape-Aware CUDA Kernel for Packed Variable-Block Sparse-Dense Matrix Multiplication

Cohen Brunner Thomas
Independent Research Project

21 August 2026

Abstract

Sparse-dense matrix multiplication (SpMM) is commonly implemented using scalar sparse formats or fixed-size block formats. Both choices are imperfect when a matrix naturally consists of dense blocks whose row and column dimensions vary independently: scalar expansion discards block structure, while global padding introduces storage and bandwidth overhead. This paper presents **VarBlockSpMM**, an FP32 CUDA implementation of the non-transpose product $C = AB$ for a packed variable-block sparse row (VBSR) matrix A and dense column-major panels B and C . Block heights and widths independently take values in $\{8, 16, \dots, 64\}$, and no global block-size padding is required.

The implementation combines row-owned output assignment, right-hand-side-width specialization, per-thread instruction-level parallelism, cooperative staging of reusable B tiles, and a measured row-shape dispatch. For 32-column panels, tall rows use asynchronous double buffering while short rows retain a lower-overhead single-buffer kernel. The direct path requires no atomics or execute-time temporary workspace. It is evaluated against a persistent scalar-CSR cuSPARSE baseline and a persistent slot-grouped cuBLAS baseline over 128 synthetic regimes and five deterministic generator seeds on one NVIDIA GeForce RTX 5060 Ti. Each of the 640 workload and seed pairs contains 20 measured iterations after five warmups. The final direct method beats both persistent library baselines in all 640 paired instances. The complete grid is repeated as five independent seed replicates, and the across-replicate mean geometric speedup with its standard error is 4.988 ± 0.021 times over cuSPARSE and 4.222 ± 0.013 times over grouped cuBLAS, showing that the effect is stable under repetition. Cluster bootstrapping over workload configurations gives geometric-mean speedups of 4.988 times (95% interval 4.463 to 5.554) over cuSPARSE and 4.222 times (4.057 to 4.388) over grouped cuBLAS. Relative to the earlier four-accumulator direct kernel, successive wider-ILP and staging changes provide a 1.46 times geometric-mean speedup over the complete grid, and the final RHS-32 double-buffer refinement adds 1.11 times over its 32-case slice. Profiling links the improvement to reduced redundant loads and reduced long-scoreboard stalls. These results establish a strong single-device proof of concept, but not a general performance claim: multi-architecture testing, real application matrices, and independent process-level repetitions remain necessary.

1 Introduction

Many hierarchical, finite-element, graph, and scientific operators contain small dense submatrices embedded in a sparse global structure. When these submatrices share a single size, block sparse row (BSR) storage exposes useful locality. Real partitions, however, often yield blocks with different heights and widths. Converting such a matrix to scalar CSR loses explicit block structure, rounding every block to one maximum dimension wastes storage and arithmetic, and dispatching a separate dense GEMM for every block pays substantial scheduling overhead.

VarBlockSpMM studies the narrower problem

$$C = AB,$$

where $A \in \mathbb{R}^{M \times K}$ is divided into variable-height block rows and variable-width block columns, while $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are dense column-major panels. The implementation supports FP32, non-transpose multiplication and $N \in \{8, 16, 32, 64\}$. Its design question is: **can a direct, workspace-free CUDA kernel retain the compactness of variable blocks while outperforming reasonable persistent library compositions across a controlled range of irregularity, degree, locality, and panel width?**

This work makes four engineering contributions:

1. A validated packed VBSR representation with independent row and column partitions and 64-bit scalar/value offsets.
2. A race-free row-owned kernel family that specializes only panel width while retaining variable block dimensions as runtime loop bounds.
3. A profiler-guided sequence of optimizations: wide accumulation to reuse each A value, cooperative shared-memory staging of B , and selective asynchronous double buffering for tall RHS-32 rows.
4. A reproducible comparison against persistent cuSPARSE and grouped-cuBLAS constructions, accompanied by correctness, sanitizer, timing, and hardware-counter evidence.

The contribution should be interpreted as an implementation and empirical case study. The associated prior-art search found no directly matching packed variable-row/variable-column FP32 SpMM interface, but it was scoped rather than exhaustive and is not a patentability or universal novelty claim.

2 Background and Related Work

GPU SpMM research has long emphasized that workload structure and row irregularity determine the useful mapping of sparse work to warps and thread blocks. Yang, Buluç, and Owens derive general GPU SpMM design principles and demonstrate row-splitting and merge-based approaches for scalar sparse matrices [1]. More recent systems exploit block structure and tensor cores, but generally target fixed or structured block layouts rather than the independently variable blocks studied here [2].

NVIDIA cuSPARSE provides SpMM for several sparse formats. Its Blocked-ELL path uses a block size belonging to the format and is tuned for fixed power-of-two blocks [3], and it does not directly encode a matrix in which every block row and column may have a different extent. NVIDIA cuBLAS provides grouped batched GEMM with per-group matrix dimensions [4]. This is a closer building block, but independent GEMMs that update the same output region cannot simply execute together without respecting accumulation dependencies. VarBlockSpMM therefore uses a slot-split organization as its cuBLAS baseline: blocks at the same ordinal position in different block rows form non-overlapping grouped calls, and slots execute sequentially with accumulation into C .

The immediate motivating application is adaptive construction of hierarchical H^2 matrices. Boukaram et al. describe a non-uniform batched BSR product and split it into a bounded number of non-uniform batched GEMM kernels in the absence of a directly suitable GPU implementation [5]. VarBlockSpMM targets the underlying packed variable-block operation rather than the full hierarchical construction algorithm.

Variable blocking predates GPU SpMM. Vuduc and Moon studied variable and unaligned block structure for sparse matrix-vector multiplication, observing that VBR exposes irregular rectangular dense regions but prevents the fixed inner-loop unrolling available to BCSR [6]. OSKI likewise defines VBR as independent logical row and column partitions with nonzero blocks stored in block-row order [7]. VarBlockSpMM adopts this classical structural model but addresses a dense multi-column right operand on CUDA.

Among GPU scalar-CSR systems, GE-SpMM caches sparse rows and merges warps to reduce redundant loads while avoiding conversion overhead [8]. Yang et al. instead emphasize merge-based load balancing and instruction-level parallelism [1]. Tensor-core SpMM methods, including the unstructured kernel of Li et al., trade preprocessing or structured packing for dense MMA throughput [2]. These methods solve complementary regimes: the present work assumes dense, semantically meaningful variable blocks and does not claim superiority for scalar sparsity. A direct experimental comparison is therefore deferred until real matrices and compatible format-conversion policies are included.

Asynchronous global-to-shared copies are a natural mechanism for hiding the latency of reusable dense-panel tiles. NVIDIA documents that these operations decouple transfer initiation from completion and can overlap data movement with computation, and hardware support is available from compute capability 8.0 for suitably aligned small copies [9]. The final RHS-32 tall-row kernel applies this mechanism conservatively, after profiling showed that a synchronous staged design had shifted its principal cost from long-scoreboard stalls toward block barriers.

3 Data Representation and Semantics

Let block-row boundaries be $r_0 = 0 < r_1 < \dots < r_R = M$ and block-column boundaries be $c_0 = 0 < c_1 < \dots < c_S = K$. Block row i has height $h_i = r_{i+1} - r_i$, and block column j has width $w_j = c_{j+1} - c_j$. Every stored block A_{ij} is a dense $h_i \times w_j$ column-major array. Its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij}B[c_j : c_{j+1}, :].$$

The host format stores block-row pointers, column-block indices, scalar row/column offsets, value offsets, and tightly packed values. Validation checks monotonic offsets, legal dimensions, in-range indices, and agreement between metadata and the value buffer. Device ownership is separated from execution planning: **Matrix** owns immutable GPU storage, while **Plan** contains a non-owning view and immutable row-classification metadata. Consequently, the matrix must outlive its plan.

Packing each block at its actual dimensions avoids the capacity cost of padding all blocks to 64×64 . The tradeoff is that address generation and loop trip counts remain dynamic inside the kernel. The format uses 64-bit scalar and value offsets to avoid limiting aggregate matrix or packed-value size through 32-bit addressing metadata.

4 Kernel Design

4.1 Row ownership

One cooperative thread array (CTA) owns one block row, and its threads cover disjoint elements of that row's output panel. Because no two CTAs update the same output scalar, the kernel needs neither atomic operations nor partial-output reduction storage. Each CTA iterates through the blocks of its row and accumulates their contributions before writing its owned elements.

The dispatch specializes on N , not on the runtime block dimensions. RHS 8 uses a scalar mapping with one output element per thread. For RHS 16 and 32, a thread accumulates eight output columns for one local row, while RHS 64 uses sixteen. Reusing one loaded A value across multiple independent accumulators reduces repeated sparse-value traffic and exposes independent fused multiply-add instructions, at the cost of registers and reduced thread-level parallelism. Measurements rejected wider mappings where that tradeoff became unfavorable.

4.2 Cooperative staging

For RHS 32 and 64, all local output rows in a CTA consume the same $w_j \times N$ slice of B . Direct global loading redundantly requests these values for each local row. The staged kernels cooperatively load the active B slice into shared memory once per block, then reuse it across local rows. A shared leading dimension of 65 floats separates neighboring columns and avoids the worst power-of-two bank-alignment pattern. The largest logical RHS-64 tile contains 64×64 floats, or 16 KiB.

The compiled synchronous staged kernels use 8,320 bytes of shared memory and 56 registers per thread for RHS 32, and 16,640 bytes and 64 registers per thread for RHS 64, without local-memory spills. RHS 16 remains on direct global loads because its smaller reuse opportunity did not repay synchronization overhead.

4.3 Shape-aware asynchronous pipeline

RHS-32 rows are classified once during matrix construction. Rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use 256 threads and two shared tiles. While the CTA computes on tile t , per-thread asynchronous copies initiate loading of tile $t + 1$. A pipeline wait and CTA barrier at the block boundary establish visibility and prevent buffer reuse before computation finishes.

Splitting a mixed-height matrix into two kernel launches is enabled only when its mean block degree is at least eight. At lower degree, measurements showed that the second launch costs more than shape separation saves, and homogeneous matrices already require only their applicable launch. The final tall-row RHS-32 kernel uses 48 registers per thread and 16,640 bytes of shared memory, while its 128-thread companion uses 54 registers and 8,320 bytes.

The same policy is deliberately not applied to RHS 64. Two full RHS-64 tiles require roughly 32.5 KiB per CTA. In the representative 4,096-row case, that allocation reduced residency and regressed median time from 2.891 ms to 4.419 ms. The accepted design therefore retains the single-buffer RHS-64 kernel.

5 Baselines and Experimental Method

Two persistent baselines separate the proposed representation from readily available library strategies.

Scalar CSR cuSPARSE. Dense blocks are expanded once into scalar CSR. The plan owns its CSR arrays, descriptors, library handle, and workspace, all of which are reused across executions. It uses 32-bit zero-based row/column indices, FP32 values and compute, column-major dense matrices, non-transpose operands, $\alpha = 1$, $\beta = 0$, and `CUSPARSE_SPMM_ALG_DEFAULT`. Descriptor creation and CSR expansion are outside the timed region, and the library workspace is persistent.

Slot-grouped cuBLAS. Packed blocks are grouped by identical (h_i, w_j) shape within each nonzero slot. One `cublasSgemmGroupedBatched` call is issued per slot, outputs within a call do not overlap, and later slots use $\beta = 1$ to accumulate. Device pointer arrays for changing B and C addresses are refreshed on the execution stream and this required marshaling is included in timing.

The grouped-cuBLAS baseline uses FP32 `cublasSgemmGroupedBatched` with non-transpose column-major operands, $\alpha = 1$, and $\beta = 0$ for the first slot and $\beta = 1$ thereafter. Packed values, the cuBLAS handle, shape groups, and A -pointer arrays are persistent. Execute-dependent B - and C -pointer arrays are rebuilt and copied asynchronously on the tested stream inside the timed region. It allocates no reported external workspace, but executes one grouped call per block-row slot.

The publication grid contains 128 workloads: 4,096 block rows and columns, mean degrees $\{2, 4, 8, 16\}$, RHS widths $\{8, 16, 32, 64\}$, four block-size distributions (uniform, low variance, high variance, and bimodal), and local or random column selection. The complete grid is repeated for seeds $\{1, 2, 3, 4, 5\}$, producing 640 workload and seed pairs with 2,560 method rows. Each point records 20 measured iterations after five warmups. GPU-event median and p95 times are reported separately from synchronized host median and p95 times. Construction and host-to-device input transfer are excluded.

The generator draws row and column block dimensions independently. “Uniform” means a constant size of 32, “low variance” samples uniformly from $\{24, 32, 40\}$, “high variance” samples from $\{8, 16, 24, 32, 40, 48, 56, 64\}$, and “bimodal” samples from $\{8, 16, 48, 64\}$. Every block row contains exactly the requested degree, with columns sampled without replacement and then sorted. Random locality samples over all block columns. Local locality samples cyclic offsets within $\pm d$ block columns of the row index, where d is degree. Stored block values are independent uniform draws on $[-1, 1]$. The timed dense input is constant 0.01, isolating structure-driven performance from input values.

The environment was an NVIDIA GeForce RTX 5060 Ti 16 GB, CUDA 13.4.46, driver 610.57.01 with CUDA UMD 13.3, MSVC 19.44, Release configuration, native architecture, and fast FP32 math. Profiler-instrumented durations are not mixed with unprofiled benchmark timings.

Correctness is checked against a double-accumulating CPU reference using a 64-case matrix spanning all RHS widths, distributions, degrees $\{1, 4, 8, 16\}$, both locality patterns, and non-default streams. The checked-in Release test passed on 21 August 2026. The project also records successful memcheck and racecheck runs with zero findings. The CPU reference accumulates in double precision. A result fails if it contains NaN/Inf, if maximum absolute error exceeds 5×10^{-4} , or if relative ℓ_2 error exceeds 5×10^{-5} . Correctness inputs are independent uniform draws on $[-1, 1]$, rather than the constant timing input.

5.1 Statistical methodology

All comparisons are paired by block-row count, degree, block-size distribution, locality, RHS width, and matrix seed. For comparator (b) and the direct method (d), the primary effect is the log speedup

$$y_{c,s} = \log(t_{b,c,s}/t_{d,c,s}),$$

where (c) indexes the 128 workload configurations and (s) indexes five matrix seeds. Geometric-mean speedup is $\exp(\bar{y})$. Log ratios are used because performance effects are multiplicative and because exchanging baseline and

Table 1: Distribution summary of paired baseline/direct GPU-median ratios over 640 workload and seed instances (128 configurations, five generator seeds). Intervals cluster-resample configurations, and values above one favor VarBlockSpMM.

Comparator	Geom. mean	95% interval	Minimum	Wins
Earlier scalar direct	2.796	[2.480, 3.126]	0.912	572/640
Scalar-CSR cuSPARSE	4.988	[4.463, 5.554]	1.607	640/640
Slot-grouped cuBLAS	4.222	[4.057, 4.388]	2.671	640/640

candidate then changes only the sign. Win counts and the minimum paired ratio accompany every aggregate so a geometric mean cannot conceal a systematic losing regime.

Two complementary uncertainty estimates are reported, because the design contains two distinct sources of variation.

Repeated-seed error. The complete 128-configuration grid is executed five times, once per generator seed, and each execution is treated as one independent replicate of the whole experiment. Let $\bar{y}_s$ be the mean log speedup over the 128 configurations of replicate (s). The repeated-seed estimate is the across-replicate mean $\bar{y} = \frac{1}{5} \sum_s \bar{y}_s$ with standard error

$$\text{SE}(\bar{y}) = \sqrt{\frac{1}{5 \cdot 4} \sum_{s=1}^5 (\bar{y}_s - \bar{y})^2}.$$

Reported values are back-transformed to the multiplicative scale as $\exp(\bar{y})$, with the corresponding interval running from $\exp(\bar{y} - \text{SE})$ to $\exp(\bar{y} + \text{SE})$. This quantifies how much the headline effect moves when the entire experiment is repeated on a fresh matrix instance, holding the configuration grid fixed.

Configuration-population error. Uncertainty over the designed workload population is estimated with a deterministic 10,000-replicate cluster bootstrap (seed 20260821). Each workload configuration is one resampling cluster, all five matrix seeds remain together, and their log ratios are averaged within the cluster. Resampling the 128 clusters therefore avoids treating 640 correlated workload and seed pairs as independent. Percentile 2.5 and 97.5 quantiles form the reported 95% interval. This interval measures sensitivity to the tested configuration population. It does *not* measure iteration-level timing noise, cross-process variation, real-matrix sampling, or GPU-to-GPU variation. No null-hypothesis significance tests are used: effect sizes, intervals, minima, and paired win counts answer the engineering question more directly. The complete calculation is implemented by `scripts/analyze_publication.ps1`.

6 Results

6.1 End-to-end regime sweep

Across five deterministic generator-seed sweeps, the final hybrid direct path beats both persistent library baselines in all 640 paired instances. A seed changes block dimensions, column selections, and stored values, so it produces a fresh matrix instance rather than an independent timing repetition of the same matrix. Against the earlier scalar direct kernel the final path wins 572 of 640 comparisons, and the exceptions concentrate in small RHS-8 cases where the accepted mapping is intentionally unchanged.

Table 1 reports the aggregate, its configuration-clustered interval, the observed minimum, and paired win count. Seeds remain grouped inside configuration clusters rather than being treated as independent timing replicates. These intervals characterize the designed configuration population, not raw timing noise, because the harness retains medians and p95 values rather than the 20 individual timings.

Across all configurations and seeds, the cluster-bootstrap geometric means (95% intervals) are 4.988 times [4.463, 5.554] over cuSPARSE, 4.222 times [4.057, 4.388] over grouped cuBLAS, and 2.796 times [2.480, 3.126] over the earlier scalar kernel. The library-baseline intervals remain well above one, while the reported minima separately establish that every tested library comparison is a win.

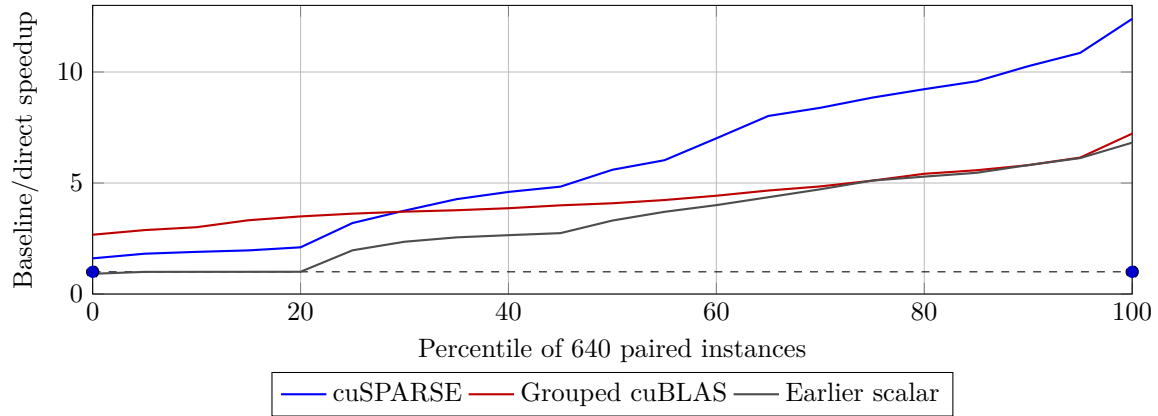


Figure 1: Empirical speedup distributions over all 640 paired workload and seed instances, plotted at five-percentile increments. The dashed line denotes parity. Both library distributions remain above it, while the earlier scalar kernel has near-parity RHS-8 cases.

Table 2: Repeated-seed replication. Each row is one complete execution of the 128-configuration grid under a different generator seed. The final row reports the across-replicate mean geometric speedup with its standard error, computed in log space and back-transformed.

Replicate	vs. cuSPARSE	vs. grouped cuBLAS	vs. earlier scalar
Seed 1	4.922	4.187	2.779
Seed 2	4.972	4.212	2.788
Seed 3	4.993	4.209	2.792
Seed 4	5.047	4.261	2.818
Seed 5	5.008	4.242	2.802
Mean $\pm$ SE	4.988 ± 0.021	4.222 ± 0.013	2.796 ± 0.007
Interval	[4.967, 5.009]	[4.209, 4.235]	[2.789, 2.802]

6.2 Repeated-seed replication

Because each seed re-executes the entire 128-configuration grid, the five sweeps form a genuine replication experiment at the level of the whole design. Table 2 lists the per-replicate geometric-mean speedup and the across-replicate mean with its standard error, computed as described in Section 5.1.

Figure 2 displays the same quantities graphically. Each panel uses its own vertical scale, because a shared axis would compress the replicate spread to less than a line width and hide precisely the quantity the figure exists to show. The axis range of every panel spans roughly 3% of its own mean.

The replicate-to-replicate spread is small. The log-scale standard deviation across the five replicates is 0.0093 for cuSPARSE, 0.0068 for grouped cuBLAS, and 0.0053 for the earlier scalar kernel, which corresponds to roughly a 0.5 to 1% multiplicative variation in the headline geometric mean. The resulting standard errors are two orders of magnitude smaller than the cluster-bootstrap intervals in Table 1. The interpretation is direct: essentially all of the reported uncertainty comes from which workload configurations are in the grid, not from which matrix instance a seed happens to draw. Repeating the experiment on new matrix instances reproduces the headline effect closely.

Two caveats bound this claim. First, the five replicates vary the matrix instance but were collected within a single process-level sweep per seed, so the standard error does not absorb cross-process, clock, or thermal drift. Second, the win counts against the earlier scalar kernel do vary across replicates (111 to 118 of 128), confirming that the near-parity RHS-8 cases sit close enough to one that instance selection flips individual comparisons even though the aggregate is stable.

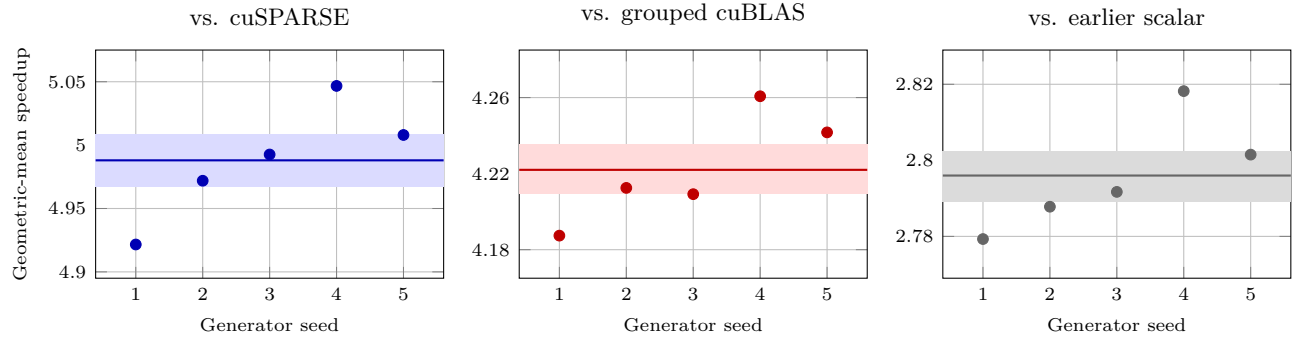


Figure 2: Repeated-seed replication error. Markers are the per-replicate geometric-mean speedups from Table 2, each obtained by running the complete 128-configuration grid under one generator seed. The solid line is the across-replicate mean and the shaded band is that mean plus and minus one standard error. Note the suppressed zero and the narrow, panel-specific vertical ranges: all five replicates of every comparator fall within about 1% of their mean, so the replication error is far smaller than any of the reported effects.

6.3 Exploratory ablation of the optimization sequence

The optimization sequence below uses the development sweeps `data/regime_map.csv`, `data/regime_map_wide_ilp.csv`, `data/regime_map_staged.csv`, and `data/regime_map_double_buffered.csv`: one generator seed, 1,024 block rows/columns, three warmups, and seven measured iterations. These data selected kernel policy and are reported as exploratory ablations, not as confirmatory evidence. Geometric means treat every grid point equally and are not weighted to represent a production workload distribution.

Change	Compared with	Evaluated slice	Geometric-mean median speedup	Additional observation
Wider per-thread ILP	Four-accumulator direct kernel	All 128 cases	1.24×	RHS 16: 1.21×, RHS 32: 1.33×, RHS 64: 1.43×
Cooperative <i>B</i> staging	Wider-ILP kernel	All 128 cases	1.18×	Every RHS-32 and RHS-64 case improved
Wider ILP + staging	Four-accumulator direct kernel	All 128 cases	1.46×	Direct path retained 128/128 library-baseline wins
RHS-32 double buffer + shape dispatch	Single-buffer staged kernel	32 RHS-32 cases	1.11×	All degree-8 and degree-16 cases improved

Longer representative measurements provide scale beyond the short sweep:

4,096-row workload	Final/accepted kernel	Comparison	Median result
Bimodal, random, degree 16, RHS 64	2.895 ms staged	4.887 ms wider-ILP	1.69× faster
Bimodal, random, degree 16, RHS 32	0.468 ms double-buffered	0.549 ms staged	1.17× faster

For the RHS-64 workload, the earlier four-accumulator result was 8.430 ms, and the cumulative progression to 2.895 ms is 2.91×. These representative figures answer an optimization question, but they should not substitute for full long-duration reruns of every regime.

6.4 Profiling evidence

Profiling of the original difficult RHS-64/high-degree kernel showed 99.54% achieved occupancy but only 0.48 eligible warps per scheduler per cycle. In that pass 71.5% of scheduler cycles had no eligible warp, and long-scoreboard waits accounted for 90.1% of the average issue interval. DRAM throughput was 56.93% of peak. Thus the limiting symptom was load latency rather than insufficient occupancy, register spilling, shared-memory spilling, or saturation of peak bandwidth.

Wider accumulation reduced redundant A traffic and raised useful work per load. Cooperative staging then reduced the representative long-scoreboard share from 78.0% to 37.1%, increased eligible warps per scheduler from 0.41 to 0.75, and raised achieved occupancy from 49.6% to 64.7%. Barrier waiting became the dominant residual cost at 36.4% of the issue interval.

For the final RHS-32 tall-row kernel, Nsight Compute confirms native asynchronous global-to-shared loads. The replay-instrumented pass reports 80.4% achieved occupancy, 1.42 eligible warps per scheduler, 47.3% of cycles with no eligible warp, 81.1% SM throughput, and 68.1% DRAM throughput. These counters support the proposed mechanism, but replay-instrumented time is excluded from benchmark speedups.

7 Discussion

The results illustrate three broader lessons. First, retaining block structure can be valuable even when block dimensions are not compile-time constants. Specializing a small orthogonal dimension, namely the dense panel width, captures much of the benefit without a combinatorial family of block-shape kernels. Second, occupancy alone did not diagnose the slow kernel: high occupancy coexisted with few eligible warps and dominant dependency stalls. Third, an optimization that is sound in isolation need not be universal. Double buffering helped tall RHS-32 rows, harmed RHS 64 through shared-memory residency pressure, and was not worth a second launch for low-degree mixed shapes.

The direct approach also has operational advantages. It owns outputs by row, so it avoids floating-point atomics and their scheduling-dependent accumulation order. It allocates no execute-time workspace, and `execute` remains asynchronous on a caller-provided CUDA stream. Conversely, plans rely on immutable matrix metadata, support only four panel widths, and currently expose only FP32 non-transpose operation. These choices make the implementation focused rather than general-purpose.

8 Threats to Validity and Limitations

The principal limitations are experimental.

- **One GPU and software stack.** Results may change with memory hierarchy, shared-memory capacity, scheduler behavior, library versions, or architecture.
- **Synthetic matrices.** The generators control block-size distribution and column locality, but they do not establish performance on matrices from a real application.
- **Summary-only timing retention.** The publication grid uses five seeds and 20 measured iterations after five warmups, but stores only median and p95 summaries. Raw-trial confidence intervals and distributional diagnostics therefore cannot be reconstructed, and each seed was collected in only one process-level sweep.
- **Restricted operation.** Only FP32, non-transpose A , column-major dense panels, block dimensions 8 to 64 in steps of eight, and RHS widths 8 to 64 are supported.
- **Baseline scope.** The baselines are credible persistent library compositions, but they are not an exhaustive comparison with research kernels, alternative reorderings, custom fixed-block ensembles, or application-specific approaches.
- **Useful-GFLOP accounting.** Reported throughput counts arithmetic from stored dense blocks. It is useful for comparisons within this study but does not measure semantic sparsity inside a nominally dense stored block.
- **Novelty scope.** Failure to identify a directly matching implementation in a scoped search is not proof that none exists.

Accordingly, the defensible conclusion is that VarBlockSpMM is a well-supported single-GPU prototype whose design wins the tested regime map, not that it is universally faster than cuSPARSE or cuBLAS.

9 Reproducibility

Artifact contents. The repository requires CMake 3.25 or newer, CUDA 13.4, and an MSVC host compiler with C++26 draft support. On Windows:

```
scripts\build.ps1
build\Release\vbsr_benchmark.exe `
  --rows 4096 --degree 8 --rhs 32 `
  --distribution high --locality random `
  --warmup 10 --reps 50 --seed 1
scripts\run_grid.ps1 -Rows 4096 -Reps 20 -Warmup 5
```

The five primary publication sweeps are stored under `data/publication/`, and `scripts/analyze_publication.ps1` reproduces the per-seed paired ratios in Table 1 together with the repeated-seed mean and standard error in Table 2. The earlier exploratory sweeps are stored in `data/regime_map.csv`, `data/regime_map_wide_ilp.csv`, `data/regime_map_staged.csv`, and `data/regime_map_double_buffered.csv`. Nsight Compute exports are retained separately under `data/`, and exact profiling commands are documented in `docs/profiling.md`. A stronger replication should retain every raw timing, repeat configurations across independent process launches, report confidence intervals on paired log-ratios, and test multiple GPU architectures.

Publication acceptance criteria. Before making an architecture-independent performance claim, the artifact should add (i) at least three GPU generations, (ii) application-derived VBR matrices with a documented partitioning procedure, (iii) independent process repetitions, (iv) raw per-repetition timings with confidence intervals, and (v) comparisons to compatible research kernels or a documented reason they cannot represent the same operation. The present artifact meets the implementation-reproducibility requirement but not these external-validity criteria.

10 Conclusion

VarBlockSpMM demonstrates that packed variable-block SpMM can be mapped efficiently to CUDA without global block padding, atomic output updates, or execute-time temporary storage. The central mechanism is not a single universal kernel but a restrained, measured policy: specialize dense-panel width, reuse sparse values through independent accumulators, stage dense-panel tiles only when reuse justifies synchronization, and double-buffer only the row shapes and degrees that repay its resource cost. Across 128 synthetic configurations and five deterministic generator seeds on the tested RTX 5060 Ti, the final implementation beats persistent scalar-CSR cuSPARSE and slot-grouped cuBLAS in all 640 paired matrix instances and beats its scalar direct predecessor in 572 of 640. Repeating the entire grid once per seed gives an across-replicate mean geometric speedup of 4.988 ± 0.021 over cuSPARSE and 4.222 ± 0.013 over grouped cuBLAS, so the headline effect is reproducible across matrix instances. Generator seeds vary matrix instances rather than supplying independent timing repetitions of a fixed matrix. The next scientific step is not further headline optimization, but broader validation across hardware, application matrices, and independent process launches.

References

- [1] C. Yang, A. Buluç, and J. D. Owens, “Design Principles for Sparse Matrix Multiplication on the GPU,” *Euro-Par 2018*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] G. Li et al., “High Performance Unstructured SpMM Computation Using Tensor Cores,” 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA, “cuSPARSE Generic API: `cusparsespm` and Blocked-ELL,” CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cusparses/>
- [4] NVIDIA, “cuBLAS `cublas<t>gemmGroupedBatched`,” CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgroupedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li, “Adaptive Sketching Based Construction of H^2 Matrices on GPUs,” *IPDPSW 2025*, 2025. <https://arxiv.org/abs/2506.16759>
- [6] R. Vuduc and H.-J. Moon, “Fast Sparse Matrix-Vector Multiplication by Exploiting Variable Block Structure,” *HPCC 2005*, 2005. doi:10.1007/11557654_91.

- [7] Berkeley Benchmarking and Optimization Group, “OSKI Variable Block Row Format,” 2007. https://bebop.cs.berkeley.edu/oski/html/group__MATTYPE__VBR.html
- [8] G. Huang, G. Dai, Y. Wang, and H. Yang, “GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks,” *SC20*, 2020. doi:10.1109/SC41405.2020.00076.
- [9] NVIDIA, “Asynchronous Data Copies,” *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>